Naming Disciplines for Generated Knowledge: the κ-Correspondence

Behnam Sour, MD
Orthopaedic Surgery, Shafa Yahyaeian Hospital, Tehran · Bone & Joint Reconstruction Research Center, IUMS
ORCID 0009-0002-3176-1956

Working paper v1 · 2026-06-11 · in preparation · not peer-reviewed.
Third paper of the series; companion to [1, 2]. Artifact: one self-contained Lean 4 file (no `mathlib`, no `sorry`, one command), an exhaustive/randomized Python suite, and corpus instrumentation, shipped as ancillary files.

Abstract

A content-addressed rule closure stores the least fixpoint $K(S)$ of finitary typed rules over a seed S, every part identified by a recursive hash of a committed projection κ of its content. Companion papers fixed one discipline (κ_0: kind, payload, premise addresses) and proved its growth and deletion theory. This paper makes the discipline the parameter. We prove a *κ-correspondence*: each storage guarantee is a functional dependency — a *determination* — between features of the committed projection. Unification of records, deduplication scope, address stability under growth, and confirmability by an outside observer are one statement instantiated at four feature pairs (the Determination Theorem; all four schemas and both corollaries kernel-checked, axiom-free, in a self-contained Lean 4 development). Two consequences. First, deduplication and confirmation-of-content are the same predicate read at two observer stations: the storage benefit and the privacy attack known from production cloud deduplication are typed as one determination, and the dedup/audit/deniability erasure triangle is tight, all three two-property vertices achievable by explicit policies. Second, the recursive clause does not reduce to the flat framework: committing premises into identity is *necessary* for exact record-local deletion (necessity converse; kernel axioms `[propext]` only), and renouncing it forces the provenance regime, the blow-up conserved between address parts and derivation records (checked $n = 3..8$: $1, 5, 16, 42, 99, 219$). The tag dial separates three independent axes — mutability, entropy, granularity — and gives a formal account of a deployed dilemma: Nix's input-addressed versus content-addressed derivations. The price of deniability is measured on a natural-language corpus: unsalted, duplicate mass 0.45% and source-aware confirmation at $2^{13.07}$ hashes; salted at λ=128, information-theoretic deniability and zero deduplication. Proofs, the exhaustive/randomized Python suite, and the one-command Lean development ship as ancillary files.

1 Introduction

Content-addressed stores of *generated* knowledge are everywhere: version control and Merkle-DAG storage name blobs by their hash; build systems name artifacts by hashes of recipes or of outputs; deduplicating cloud storage names files by their digests; content-addressed languages name code by the hash of its syntax tree. Each such system fixes, usually implicitly, a *naming discipline*: a choice of which features of a part's content are committed into its address. Everything else about the store — what deduplicates, what stays stable as the store grows, what an outsider can confirm, what can be deniably erased, and what deletion costs — follows from that choice.

The companion papers [1, 2] fixed one discipline, κ_0 (commit the kind, the payload, and the addresses of the premises), and developed its theory: growth is conservative, incremental, history-independent, and continuous; deletion satisfies a cone theorem with a characterized anomaly; a monotonicity boundary localizes CALM in the rule format. This paper asks the prior question: *why these guarantees from this discipline?* The answer in the literature is folklore, distributed by community. Security knows the deduplication/confidentiality tradeoff and the confirmation side-channel; database theory knows monotonicity boundaries, deletion-propagation complexity, and provenance bookkeeping; the data-structures literature knows history independence; build systems debate input- versus content-addressing in RFCs. Each community owns one edge of the picture and a local vocabulary for it; none states the common mechanism.

We make the discipline the parameter. An identity scheme is a committed projection π_κ of content; a guarantee is a determination — a functional dependency between features of that projection; and the correspondence says the two lists are the same list. Concretely:

1. **The frame** (§2): disciplines κ as committed projections; the guarantee vocabulary as predicates over stores and observers.
2. **The Determination Theorem** (§7): each guarantee holds iff a determination $\mathrm{Det}(k, x)$ holds on π_κ, uniformly across four feature pairs; all four schemas and both corollaries kernel-checked and axiom-free.
3. **The recursive clause does not reduce** (§3): committing premises into identity is *necessary* for exact record-local deletion — the converse of the companion papers' design choice (kernel axioms `[propext]` only) — with a dichotomy, inscription or the provenance regime, and a conservation law equating the blow-up paid in address parts with the one paid in derivation records.
4. **The erasure triangle and the two-observer identity** (§4): $\mathrm{DEDUP} \wedge \mathrm{AUDIT} \Rightarrow \neg\mathrm{DEN}$ beyond min-entropy; all three two-property vertices achievable by explicit policies; deduplication and confirmability one predicate at two observer stations — the typing of a known production attack.
5. **The tag dial** (§5): tag mutability, tag entropy, and granularity are independent axes; the dial's endpoints give a formal account of Nix's input- versus content-addressed derivations.
6. **The price of deniability, measured** (§6): on a natural-language corpus, what salting costs in deduplication and buys in confirmation resistance, with the λ exchange rate explicit.
7. **The verification ladder** (§8): every claim paper-proved; the finite claims verified exhaustively or at corpus scale in Python; the core kernel-checked in one self-contained Lean 4 file — no `mathlib`, no `sorry`, one command.

Honesty about depth, up front: the determination schemas are deliberately elementary — the dependency framework is Armstrong's [3], labeled as such — and the unification, the converse, and the kernel artifact are the contribution. §9 places every edge of the correspondence next to its community cousin; the three closest neighbors are credited inside their own sections, not buried in related work.

2 Setting: identity schemes

The companion papers fix one identity discipline and derive its consequences. This note makes the discipline the parameter.

Definition 1 (Identity scheme)**.** *Fix a feature alphabet for parts:* content *(payload bytes),* premises *(sorted child addresses),* rule*,* descriptor*,* nonce *(fresh randomness of min-entropy* $\geq \lambda$*),* epoch*. An* identity scheme *is a pair* (κ, H) *of a commitment set* κ *of features and a collision-resistant hash* H*; the address of a part is* H *applied to its* κ*-projection, recursively through premises. Injectivity of addresses on committed features is the single extra-mathematical assumption, as in H4 of [1].*

Two schemes are compared throughout. $\kappa_0 = \{\text{rule}, \text{descriptor}, \text{premises}, \text{content}\}$ is the companion papers' H4: identity inscribes derivation. $\kappa' = \{\text{rule}, \text{descriptor}\}$ omits premises: all admissible bindings of one descriptor cohort collapse to *one* part — the natural "one object per cohort" design, and exactly the versioned-cohort resolution of [1, §6.2].

Under κ_0, the deletion paper proves: derivations are unique (premise inscription $\Rightarrow$ single derivation), hence deletion is *exact and record-local* — removing the upward cone, computed from each part's own stored record, coincides bit-for-bit with re-closure (the cone theorem), and DRed-style alternative-derivation accounting [4] is structurally absent. This note proves the converse direction and then characterizes the erasure face of the correspondence.

3 O1: premise inscription is necessary

Lemma 2 (Preimage coincidence)**.** *If premises* $\notin \kappa$*, then any two rule instances agreeing on the committed features but differing in premise sets produce the* identical *address. No hash collision is involved: the preimages are equal. The output is one part with at least two derivations; single derivation fails.*

Proof. Immediate from Definition 1: the address is a function of the κ-projection alone. □

The substantive question is whether multiplicity actually destroys record-local deletion, or whether some clever choice of stored record survives it. It does not survive it.

Theorem 3 (Necessity of premise inscription)**.** *Consider the* κ'*-store on a single cohort of four atoms with threshold* $\tau = 3$*. Call* r *a* representative record *if* r *is an admissible premise set retained as the part's stored children, and say* r *is* correct *if record-local cone excision under* r *agrees with re-closure on every deletion* D*. Then* ***no representative record is correct****: for every admissible* r *there is a single-atom deletion on which excision diverges from ground truth — and always by* over-deletion *(excision removes the part while an alternative derivation survives).*

Proof. Any admissible r has $|r| \geq \tau = 3$. Pick any $a \in r$ and delete $D = \{a\}$. The survivors number $4 - 1 = 3 \geq \tau$, so re-closure retains the part (some admissible subset survives). But $r \cap D \neq \emptyset$, so record-local excision removes it. Hence every r fails, by over-deletion. □

Mechanization. Kernel-checked as `Growth.Necessity.necessity` (a bounded universal statement discharged by `decide`; axioms `[propext]` only), with the concrete face `witness_overdelete` (record $\{a_1, a_2, a_3\}$, delete a_1); exhaustively re-verified in Python over all five admissible records and all 2^4 deletions.

Corollary 4 (Deletion dichotomy at the identity layer)**.** *Exact deletion forces a choice of where derivations live. Either* (i) *premises* $\subseteq \kappa$*: identity splits where provenance differs, derivations are unique, and deletion is record-local and exact (the regime of [2]); or* (ii) *the store keeps the full derivation multiset per part and tests survival on deletion —* ***the provenance regime****: counting/DRed view maintenance [4], i.e. exactly the bookkeeping that provenance semirings make systematic [16] — which is exact (kernel-checked as* `counting_exact`*) but no longer record-local: correctness consults every alternative derivation. There is no*

third option within record-local semantics (Theorem 3). The contrast with relational deletion propagation [14, 15] is structural: their problem optimizes among alternative derivations; premise inscription removes the alternatives, so propagation under κ_0 is exact and optimization-free — key preservation [14] is the relational cousin of clause (i).

Remark 5 (Conservation of the blow-up)**.** *The descriptor-clique blow-up of [1, Prop. 10] is invariant under the choice: a cohort of n atoms costs $\sum_{m \geq \tau} \binom{n}{m}$* parts *under κ_0 and the same number of* derivation records *attached to one part under κ' (verified $n = 3..8$: $1, 5, 16, 42, 99, 219$ on both sides). The exponential cannot be deleted from the design space; it can only be moved between identity and provenance. This sharpens the companion papers' design principle:* monotone, complete storage; parsimony in views — *and now,* provenance in identity, or identity's work done again in provenance*.*

Theorem 3 is, to our knowledge, the first statement in this development that none of the prior slices contains: CALM [5, 6] speaks of monotonicity, not identity; the deletion paper proves sufficiency of inscription; the necessity direction is new. It also *explains* a design fact the companion papers record without explanation: why the versioned-cohort resolution (§6.2 of [1]) must carry a version chain — the chain is the derivation bookkeeping that Corollary 4(ii) makes mandatory once premises leave the preimage.

4 The erasure triangle

The deletion paper's retention analysis tabulates policies; this section proves the table is *tight*. Fix the κ_0-discipline for live content and consider what a deletion policy retains about removed parts.

Definition 6 (The three properties)**.** ***DEDUP****: addresses are a deterministic function of content-determined features (re-ingesting equal content yields the equal address).* ***AUDIT****: from retained data alone, plus a claimed preimage, anyone can verify non-interactively that a specific part existed and was removed.* ***DEN*** *(deniable erasure at level λ): in the* confirmation game*, an adversary holding the retained data and any candidate payload p decides whether p was removed with advantage at most $2^{-\lambda}$ over guessing.*

Theorem 7 (Tightness of the triangle)**.** (Impossibility.) *DEDUP $\wedge$ AUDIT $\Rightarrow \neg$DEN beyond payload min-entropy: the adversary hashes the candidate p under the public, content-determined address function and tests membership in the retained address set; the advantage is 1 for any candidate equal to a removed payload, so deniability degrades to the 2^{-H_∞} guessing bound — one dictionary lookup for published text.* (Achievability.) *Each pair is realized by a measured policy of [2]: {DEDUP, DEN} by excision or epoch re-closure (nothing retained $\Rightarrow$ nothing to audit); {DEDUP, AUDIT} by address-only tombstones (deniability only up to min-entropy, as above); {AUDIT, DEN} by salting (*nonce $\in \kappa$*): retained addresses are unverifiable without the per-part nonce, so erasure is deniable to outsiders* and *provable by selective disclosure of the nonce — at the price of DEDUP, since equal content no longer shares an address.*

Proof. Impossibility is the confirmation attack written out: DEDUP makes the address function public and content-determined; AUDIT requires the removed addresses to be retained and verification to be non-interactive; composing the two is the adversary. Achievability: the first two rows are Propositions of [2, §Erasure] (retention separates the policies; address-only tombstones); the third row follows from Lemma 2 read contrapositively — a committed high-min-entropy nonce makes the address indistinguishable from random to anyone lacking the nonce (advantage $\leq 2^{-\lambda}$), while disclosure of (p, nonce) re-derives the address and convinces a verifier. □

Remark 8 (Selective audit; placement of prior art)**.** *The third vertex converts AUDIT from public to* designated-verifier*: the deleting party can always prove a specific erasure, outsiders can verify nothing — arguably the correct posture for Art. 17 obligations. Credit where it is due: the dedup-vs-confidentiality mechanism is the convergent-encryption tradeoff [23], formalized game-theoretically as message-locked encryption [9, 25]; and the confirmation attack itself was demonstrated against production cloud deduplication by Harnik, Pinkas & Shulman-Peleg [7], with deduplication-as-access-oracle following in [8]. Corollary 16 is the* typing *of their observation; the AUDIT axis, the tightness statement, and the placement inside closure theory are what this paper adds. The mechanism is theirs; the triangle is ours.*

5 κ5: the tag dial

Definition 9 (Tagged scheme)**.** $\kappa_T = \kappa_0 \cup \{\text{tag}\}$ *with tags drawn from a set* T *and committed into the preimage; a part is a pair* $(\text{tag}, \text{content})$*, and the closure development applies unchanged at the tagged universe (the formalization is parametric in* U*; instantiate at* $T \times U$*).*

Theorem 10 (The epoch row)**.** *Under* κ_T*:* (i) Re-addability. *Content removed at tag* e *returns live on re-ingestion at any fresh tag* e' *with* $(e', u) \notin R$ *— by extensivity alone (kernel-checked,* axiom-free*, as* `Growth.Epoch.epoch_readd`*).* (ii) Instability. *The same content at different tags is a different part (*`epoch_fresh`*, axiom-free): cross-tag address stability and cross-tag deduplication fail by construction.* (iii) Locality. *Within a single tag-class, the entire* κ_0 *package of [1, 2] holds, tag-fiber by tag-fiber.*

The pairing is the content of the row: `tombstone_permanence` [2] says removal is permanent at a *fixed identity*; `epoch_readd` says the *content* returns at a fresh one. Re-addability is bought exactly by identity churn — never for free.

Remark 11 (Three independent axes of one tag)**.** *The dial's endpoints: a constant tag recovers* κ_0 *(tombstone permanence governs; no re-add); an injective-per-ingestion tag is the nonce vertex of Theorem 7 (no two ingestions ever share identity). Intermediate granularity — epochs — gives era-local deduplication with boundary re-addability. The crucial separation from* κ*3: epochs are* enumerable *(low conditional min-entropy), so mutability buys READD* without *deniability. One committed tag, three independent levers:* ***mutability*** $\Rightarrow$ *re-addability;* ***entropy*** $\Rightarrow$ *deniability;* ***granularity*** $\Rightarrow$ *deduplication scope.*

The dial, deployed. The choice this section formalizes is not hypothetical: it is the live design axis of the Nix store. Input-addressed derivations commit the *build inputs* into the output path — premises $\subseteq \kappa$, references stable before the build, signatures required for trust, rebuilds propagating through the dependency graph — while content-addressed derivations commit the *output content*, buying "early cutoff" (the dedup schema at output granularity) and signature-free verification (the confirmability clause, with the trust-model concerns that the two-observer identity predicts) at the price of eval-time-stable references [21, 22]. The correspondence is, to our knowledge, the first formal account of that documented engineering dilemma.

6 O5: the price of deniability, measured

Theorem 12 (Confirmation bound)**.** *In the confirmation game against a retained address set, an adversary making* q *hash evaluations has advantage at most* $q \cdot 2^{-(H_\infty(\text{payload}|\mathcal{A}) + H_\infty(\text{tag}|\mathcal{A}))}$*.* Proof: *union bound over queries against the unpredictability of the committed pair; hashing supplies free verification of a guess and nothing more [24]. The bound is the standard guessing bound; what follows — the instrumentation — is the contribution claimed.*

Measurement (Moby-Dick sentence corpus; pipeline of [1, §Empirics]). Extracted ingestions 8636 (of which 8631 carry a descriptor — the cohort pipeline's denominator); distinct payloads 8597; **dedup mass** 39 **ingestions** $= 0.45\%$ (the most-repeated payload is Ahab's refrain, "Hast seen the White Whale," $\times 4$). Unsalted address-only tombstones on this corpus: a source-aware adversary covers the entire payload space in 8597 hashes ($2^{13.07}$), confirming any removed sentence at $O(1)$ per candidate — **deniability** 0 **bits**, the worst case by construction (published text), which is precisely why it is the right stress test. Salted at $\lambda = 128$: per-candidate cost multiplies by 2^{128}; the price is the 0.45% of ingestions no longer unified.

Remark 13 (Exchange rate)**.** *On natural-prose corpora the salted vertex of the triangle is* nearly free*:* 128 *bits of deniability for* 0.45% *storage. The trade inverts on dedup-heavy corpora (artifact stores, blob caches), where the same construction forfeits its central economy. The triangle's binding constraint is a per-corpus measurable, and the measurement is one pass over the payload histogram.*

7 The Determination Theorem

The six rows now admit a single statement. The right abstraction is the oldest one in database theory: a *determination* — observer-knowledge k determines quantity x when k-equal events are x-equal — i.e. a functional dependency [3], transported from relations to identity schemes.

Definition 14 (Determination)**.** *For maps $k : E \to K$ and $x : E \to X$ on ingestion events,* $\mathrm{Det}(k, x)$ *holds iff* $\forall e, e' : k(e) = k(e') \Rightarrow x(e) = x(e')$*. An identity scheme is* $\mathrm{addr} = H \circ \pi_\kappa$ *with H injective on committed projections (the H4-style assumption).*

Theorem 15 (Determination Theorem)**.** *For every commitment κ, granularity g, context map c, and observer knowledge k:*

(a) ***Unification.*** $\mathrm{addr}(e) = \mathrm{addr}(e') \iff \pi_\kappa(e) = \pi_\kappa(e')$.

(b) ***Deduplication.*** *g-granular dedup* $\iff \mathrm{Det}(g, \pi_\kappa)$*: dedup holds exactly at granularities that determine the commitment. Instances: $g =$ content gives $\kappa 2$; a committed nonce defeats every g below events ($\kappa 3$); a committed epoch yields dedup within eras ($\kappa 5$'s granularity lever).*

(c) ***Stability.*** *Addresses are c-invariant* $\iff$ *π_κ is c-invariant. Instances: growth leaves derivation content fixed — the identity half of conservativity; an era bump changes the committed tag — XSTAB fails ($\kappa 5$;* `epoch_fresh` *is the pointwise face).*

(d) ***Confirmation.*** $\mathrm{Det}(k, \pi_\kappa)$ *lets observer k decide membership of any retained address set; its failure is quantified by $H_\infty(\pi_\kappa \mid k)$ (Theorem 12).*

(e) ***Recursive clause.*** *Exact record-local deletion* $\iff$ *premises $\subseteq \kappa$ — sufficiency is the deletion paper's cone theorem, necessity is Theorem 3. This clause provably does* not *reduce to (a)–(d): premises feed addresses back into their own determining features, outside the flat dependency model.*

Clauses (a)–(d) and both corollaries below are kernel-checked axiom-free *(*`Growth.Unified: unify_iff, dedup_iff, stable_iff, confirm_of_det`*); each is also instantiated on the reference implementation (*`verify_unified.py`*).*

Corollary 16 (Two-observer identity)**.** *DEDUP is* $\mathrm{Det}(\textit{content}, \mathrm{addr})$*;* $\neg$*DEN is* $\mathrm{Det}(k_{\mathrm{adv}}, \mathrm{addr})$*.* ***Deduplication and confirmability are the same predicate evaluated at two stations*** *(*`dedup_is_confirmability`*). The erasure triangle's impossibility edge follows in one line: DEDUP plus a public retained set hands the store's unification power to every content-holding adversary (*`triangle_edge`*) — the confirmation attack is a typing fact, not a clever construction.*

Corollary 17 (The correspondence, closed)**.** $\kappa 1$ = *clause (e);* $\kappa 2, \kappa 3$ = *clause (b) at content/event granularity;* $\kappa 5$ = *clauses (b)+(c);* $\kappa 6$ = *clause (d) plus retention; and* $\kappa 4$ *is the same principle applied to the* side-condition *scope* σ *instead of* κ*: firing determined by premise payloads* $\Rightarrow$ *monotonicity (*`step_mono`*, axiom-free), with seed-global features outside* σ *exactly when maximality is demanded (Theorem 11 of [1]). Guarantees split into an injectivity family, monotone increasing in* κ*, and a coarseness family, antitone — the lattice structure of [3] — with the six rows as the boundary instances.*

Remark 18 (What is old, what is new)**.** *Determinations are functional dependencies and the antitone lattice is Armstrong's closure theory [3]; this section claims no novelty there. Three things do not come from dependency theory: the* recursive clause *— addresses occurring inside their own determining features, where flat attribute models do not reach and where the development's one substantive converse (Theorem 3) had to live; the* two-observer identification*, a security reading for which dependency theory has no station; and the* min-entropy quantification *of failed determinations (§6). The schemas themselves are one-line proofs by design: the theorem's content is that every row reduces to them* except *the one that provably cannot. In an identity scheme, everything but recursion is bookkeeping; the recursion is where the theorems live.*

Remark 19 (The schema audits its substrate)**.** *Instantiating clause (a) on the reference implementation exposed a preimage-discipline subtlety: the atom constructor commits* $(kind, payload)$ *only — the descriptor argument is* not *in the preimage, so equal payloads unify across descriptor arguments. The papers' invariant that the descriptor is a payload* field *is precisely the discipline that makes this safe; an implementation violating it would acquire a history-dependent descriptor. Composites do commit the sign. The finding is recorded as a check in* `verify_unified.py`*.*

8 The verification ladder

Every claim sits on the three-rung ladder of the companion papers. *Rung 1 (paper):* all theorems above carry full proofs. *Rung 2 (executable):* the finite claims are checked by programs shipped with the paper. `verify_o1_necessity.py` verifies Theorem 3 exhaustively — five-record stores against all 2^4 deletion sets, over-deletion in every premises-uncommitted case, zero exceptions — and the conservation law for $n = 3..8$ ($1, 5, 16, 42, 99, 219$ on both sides). `verify_unified.py` re-checks all four determination schemas and both corollaries against the reference implementation (7/7), and earned its keep with a finding: the implementation's `gc.atom` commits `(kind, payload)` but not the descriptor argument, sound only under the papers' descriptor-as-payload-field invariant — schema (a) is precisely the audit that detects such gaps. `corpus_o5.py` produces the §6 numbers (8636 sentences; duplicate mass $39/8636 = 0.45\%$; source-aware confirmation $2^{13.07}$ hashes; λ=128 salting; epoch semantics demonstrated: same-era dedup, cross-era freshness, re-add). *Rung 3 (kernel):* the shared development `GrowthClosure_kappa.lean` compiles with one command on Lean 4.30.0 — exit 0, no `sorry`, no `mathlib` — and the axiom audit for this paper's namespaces, re-run on the submitted file, reads verbatim:

```
'Growth.Necessity.necessity' depends on axioms: [propext]
'Growth.Necessity.counting_exact' depends on axioms: [propext]
'Growth.Necessity.witness_overdelete' depends on axioms: [propext]
'Growth.Epoch.epoch_readd' does not depend on any axioms
'Growth.Epoch.epoch_fresh' does not depend on any axioms
'Growth.Unified.unify_iff' does not depend on any axioms
'Growth.Unified.dedup_iff' does not depend on any axioms
'Growth.Unified.stable_iff' does not depend on any axioms
'Growth.Unified.confirm_of_det' does not depend on any axioms
'Growth.Unified.dedup_is_confirmability' does not depend on any axioms
'Growth.Unified.triangle_edge' does not depend on any axioms
```

The twenty inherited declarations of the growth and deletion namespaces audit as in the companions: `conservativity`, `galois`, `descent`, `step_mono`, `step_finitary`, and the 2P/survivor lemmas axiom-free; the remainder on `[propext, Quot.sound]`; no `Classical.choice` anywhere in the development. One cosmetic linter warning (an unused variable) is the file's entire diagnostic output.

9 Related work and placement

Monotonicity boundaries. CALM and its proof [5, 6] fix the order and quantify over queries; Bloom^L [17] extends the analysis to arbitrary lattices and cross-lattice morphisms — the 2P product order of [2] is an instance of that move; monotone *queries* over CRDTs [18], free termination [19], and coordination with global knowledge [20] extend the same axis. None of these has a notion of part identity; the correspondence adds the orthogonal axis and recovers $\kappa 4$ as its CALM slice.

Deduplication security. The confirmation attack against production cloud dedup is [7]; dedup as an access oracle is [8]; the DEDUP/confidentiality edge is formalized as message-locked encryption [23, 9, 25]. Corollary 16 types these as one determination at two stations; the AUDIT vertex and the triangle's tightness are new here.

History independence. Introduced for data structures by [10] and formalized as anti-persistence by [11] — whose motivation already includes the privacy of *deleted* data, the ancestor of DEN; strong history independence is characterized by canonical representations [12]. The bridge: content addressing supplies a canonical representation for free, so the canonical serialization of [1] makes the store strongly history-independent in exactly the [12] sense — an instance gained, at the logical-state rather than memory-representation level. The systems taxonomy of erasure is [13]; the identity-layer cut (what *addresses* retain) is this paper's addition.

Deletion propagation and provenance. Complexity dichotomies for side-effect-minimizing deletion are [14, 15]; provenance semirings [16] systematize derivation bookkeeping and its incremental maintenance. The cone theorem and Corollary 4 sit exactly between them: inscription makes propagation trivial; renouncing inscription lands in the provenance regime, paying the conserved blow-up there.

Deployed identity schemes. The Nix store's input-addressed vs content-addressed derivations [21, 22] instantiate the dial at ecosystem scale (§5); Merkle-CRDTs and Git/IPFS supply the storage lineage already placed in [1]. Determinations are functional dependencies [3]; the recursive clause is the part of identity outside that flat model.

10 Limitations and scope

Five boundaries, stated plainly. *(1)* The schemas are elementary by design; a reader who wants hardness will find it only in the recursive clause and in the impossibility results inherited from [1, 2]. *(2)* Everything is conditional on the companion papers' rule format and hypotheses H1–H4; the discipline is varied, the generation model is not. *(3)* The corpus instrumentation is a single English text, illustrative rather than statistical; the exchange-rate arithmetic, not the corpus, carries the claim. *(4)* The confirmation bound is the standard min-entropy argument; the paper's addition is its placement, not its strength. *(5)* One related-work citation (build-systems early-cutoff theory) is flagged for verification at camera-ready, and the residual sweep list — Unison's content-addressed code model, formal Git/IPLD semantics, incremental chase maintenance — is recorded in the ancillary sweep document.

11 Conclusion

All six rows of the correspondence are resolved and derived from one statement: $\kappa 1$ in both directions, with the converse kernel-checked (§3); $\kappa 2/\kappa 3/\kappa 6$ by the erasure triangle (§4), whose impossibility edge re-derives as `triangle_edge`; $\kappa 4$ by the σ-column of Corollary 17; $\kappa 5$ by the tag dial (§5); the quantitative face by §6; and the whole by the Determination Theorem (§7), all four schemas and both corollaries axiom-free. What a closure store can deduplicate, what it must reveal, what it can deny, and what deletion costs are not separate engineering facts: they are one functional dependency read at different stations, with the recursive clause — premises in the address — as the one part of identity the flat framework cannot see. The discipline, not the data, is the design surface.

References

[1] B. Sour. *Monotone Growth of Content-Addressed Rule Closures.* Working paper v2, 2026 (in preparation).

[2] B. Sour. *Principled Deletion in Content-Addressed Rule Closures.* Working paper v1.1, 2026 (in preparation).

[3] W. W. Armstrong. Dependency structures of data base relationships. *IFIP Congress* 1974.

[4] A. Gupta, I. S. Mumick, V. S. Subrahmanian. Maintaining views incrementally. *SIGMOD* 1993.

[5] J. M. Hellerstein. The declarative imperative. *SIGMOD Record* 39(1), 2010.

[6] T. J. Ameloot, F. Neven, J. Van den Bussche. Relational transducers for declarative networking. *J. ACM* 60(2), 2013.

[7] D. Harnik, B. Pinkas, A. Shulman-Peleg. Side channels in cloud services: deduplication in cloud storage. *IEEE Security & Privacy* 8(6), 2010.

[8] S. Halevi, D. Harnik, B. Pinkas, A. Shulman-Peleg. Proofs of ownership in remote storage systems. *CCS* 2011.

[9] M. Bellare, S. Keelveedhi, T. Ristenpart. Message-locked encryption and secure deduplication. *EUROCRYPT* 2013.

[10] D. Micciancio. Oblivious data structures: applications to cryptography. *STOC* 1997.

[11] M. Naor, V. Teague. Anti-persistence: history independent data structures. *STOC* 2001.

[12] J. D. Hartline et al. Characterizing history independent data structures. *ISAAC* 2002.

[13] J. Reardon, D. Basin, S. Čapkun. SoK: secure data deletion. *IEEE S&P* 2013.

[14] P. Buneman, S. Khanna, W.-C. Tan. On propagation of deletions and annotations through views. *PODS* 2002.

[15] B. Kimelfeld, J. Vondrák, R. Williams. Maximizing conjunctive views in deletion propagation. *ACM TODS* (PODS line, 2011–2013).

[16] T. J. Green, G. Karvounarakis, V. Tannen. Provenance semirings. *PODS* 2007.

[17] N. Conway, W. R. Marczak, P. Alvaro, J. M. Hellerstein, D. Maier. Logic and lattices for distributed programming. *SoCC* 2012.

[18] S. Laddad, C. Power, M. Milano, A. Cheung, N. Crooks, J. M. Hellerstein. Keep CALM and CRDT on. *PVLDB* 16(4), 2022.

[19] C. Power. Free termination. *ICDT* 2025.

[20] T. Baccaert, B. Ketsman. Distributed consistency beyond queries. *PODS* 2023.

[21] E. Dolstra. *The Purely Functional Software Deployment Model.* PhD thesis, Utrecht University, 2006.

[22] T. Gérard et al. NixOS RFC 0062: content-addressed paths. 2019–.

[23] J. R. Douceur et al. Reclaiming space from duplicate files in a serverless distributed file system. *ICDCS* 2002.

[24] P. Rogaway, T. Shrimpton. Cryptographic hash-function basics. *FSE* 2004.

[25] M. Bellare, S. Keelveedhi, T. Ristenpart. DupLESS: server-aided encryption for deduplicated storage. *USENIX Security* 2013.